

Fog Detection and Road Safety Assistance using Transfer Learning

UBCA398J – Project

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications

by

DIVYA G

23BCA0113

Under the guidance of

Dr. KALAIVANI S

Associate Professor Grade 1, SCORE

School of Computer Science Engineering and Information Systems

VIT, Vellore



April, 2026

Fog Detection and Road Safety Assistance using Transfer Learning

UBCA398J – Project

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications

by

DIVYA G

23BCA0113

Under the guidance of

Dr. KALAIVANI S

Associate Professor Grade 1, SCORE

School of Computer Science Engineering and Information Systems

VIT, Vellore



April, 2026

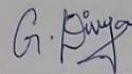
DECLARATION

I hereby declare that the UBCA398J – Project entitled “**Fog Detection and Road Safety Assistance using Transfer Learning**” submitted by Divya G , for the award of the degree of *Bachelor of Computer Applications, School of Computer Science Engineering and Information Systems* to VIT is a record of bonafide work carried out by me under the supervision **Dr Kalaivani S, Associate Professor Grade 1, SCORE, VIT, Vellore.**

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 17/04/26



Signature of the Candidate

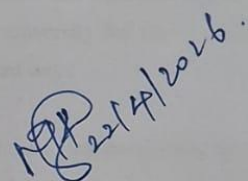
CERTIFICATE

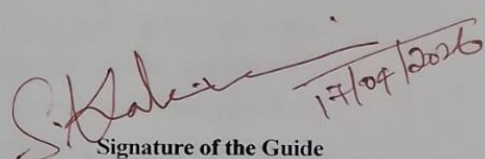
This is to certify that the UBCA398J – Project entitled “Fog Detection and Road Safety using Transfer Learning” submitted by Divya G & Reg. No:23BCA0113, SCORE, VIT, for the award of the degree of *Bachelor of Computer Applications, School of Computer Science Engineering and Information Systems*, is a record of bonafide work carried out by her under my supervision during the period, 05. 12. 2025 to 17. 04. 2026, as per the VIT code of academic and research ethics.

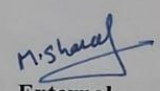
The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

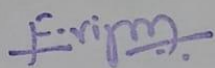
Place: Vellore

Date: 17/4/26


Internal Examiner
Examiner


Signature of the Guide


External


Head of the Department
Department of Computer Applications

Head of the Department,
Department of Computer Applications,
School of Computer Science Engineering
and Information Systems,
Vellore Institute of Technology (VIT),
(Deemed to be university under section 3 of UGC ACT, 1956)
Vellore – 632 014.

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my **UBCA398J – Project** guide **Dr. Kalaivani S, Associate Professor Grade 1**, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for **her** constant guidance, continual encouragement, in my endeavor. My association with **her** is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of **Deep Learning**.

"I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan, Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**, and Registrar **Dr. Jayabarathi T**.

My whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems, Head, Department of Computer Applications, **Dr. Vijayan E**, Project Coordinator **Dr. Ephzibah E.P**, SCORE School Project Coordinator **Dr. Thandeeswaran R**, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my project successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the project.

Place: Vellore

Date: 17/04/26

Divya G

Executive Summary

Fog is one of the major weather conditions that significantly reduces road visibility, leading to serious accidents and fatalities. Traditional sensor-based fog detection systems are expensive, difficult to maintain, and not widely available. This project proposes a Fog Detection and Road Safety Assistance System using deep learning to provide real-time, camera-based fog classification and intelligent driver guidance.

The system uses a pre-trained ConvNeXt-Base deep learning model fine-tuned on the Foggy Cityscapes dataset to classify road images into three fog density levels: No Fog, Medium Fog, and Dense Fog, achieving a test accuracy of over 90%. A vision-based fog correction engine further refines predictions using contrast analysis, edge density, sky haze estimation, and dark channel prior techniques. YOLOv8 object detection is integrated to identify vehicles, pedestrians, and road signs simultaneously.

The system was tested on a real driving video of 1,624 frames, detecting 32.9% Light Fog and 67.1% Medium Fog conditions, with an average visibility of 189 metres. Real-time safety guidance including speed recommendations, brake alerts, and lane loss warnings are overlaid on the video feed via a Heads-Up Display.

The proposed system is cost-effective, deployable on standard hardware, and offers a scalable solution for smart transportation and advanced driver assistance system.

Table of Contents

Contents	Page no.
Acknowledgement	v
Executive Summary	vi
Table of Contents	vii
List of Figures	ix
List of Tables	x
Abbreviations	xi
Symbols and Notations	xii
1. INTRODUCTION	1
1.1 Objective	2
1.2 Motivation	2
1.3 Background	3
2. PROJECT DESCRIPTION AND GOALS	7
2.1 Project Scope	7
2.2 Application Areas	7
2.3 Goals	8
2.4 Expected Outcomes	9
3. TECHNICAL SPECIFICATION	10
3.1 System Requirements	10
3.1.1 Hardware Specifications	10
3.1.2 Software Specifications	11
3.2 Justification of Specifications	12
4. DESIGN APPROACH AND DETAILS	13
4.1 Design Approach	13
4.2 Codes and Standards	16
4.3 Constraints, Alternatives and Tradeoffs	17

5. SCHEDULE, TASKS AND MILESTONES	19
5.1 Project Timeline	19
5.2 Project Timeline Summary	21
5.3 Conclusion	22
6. PROJECT DEMONSTRATION	23
6.1 Sample Codes	23
6.1.1 Deep Learning Code	23
6.1.2 Video Processing Code	33
6.2 Sample Output	38
7. RESULT	39
7.1 Dataset Description	39
7.2 Intermediate Results	41
7.2.1 Visualization Image	41
7.2.2 Preprocessing Image	41
7.2.3 Class Distribution Bar Chart	43
7.2.4 Prediction Image	43
7.3 Final Output Image	45
7.3.1. Fog Analytics Chart	45
7.4 Accuracy and Loss Graph	46
7.5 Confusion Matrix	47
7.6 Discussion	48
7.7 Conclusion	49
7.8 Limitations	50
7.9 Future Work	50
8. SUMMARY	51
9. REFERENCES	52

List of Figures

Figure No.	Title	Page No.
4.1.1	System Architecture	14
4.1.2	UML Diagram	15
5.1	Project Timeline	19
6.1	Image upload and Output video	38
6.2	Analysis Result	38
7.1	Dataset Images	39
7.2.1	Visualization Images	41
7.2.2	Preprocessed Images	42
7.2.3	Class Distribution Bar Chart	43
7.2.4	Prediction Images	44
7.3	Output Images	45
7.3.1	Fog Analytics chart	45
7.4.1	Accuracy and loss Graph	46
7.5	Confusion Matrix and Normalized	47
7.6.1	Comparative Analysis Graph	49

List of Tables

Table No.	Title	Page No.
Table 1.3.1	Literature Survey	3
Table 3.1	Hardware Specifications	11
Table 3.2	Software Specifications	11
Table 4.3.1	Tradeoff and Description	18
Table 5.2.1	Milestones and Description	22
Table 7.6	Comparative Analysis Table	48

List of Abbreviations

ADAS	Advanced Driver Assistance Systems
AI	Artificial Intelligence
AUC	Area Under the Curve
AWGN	Additive White Gaussian Noise
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DCP	Dark Channel Prior
DL	Deep Learning
FPS	Frames Per Second
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
HUD	Heads-Up Display
IoT	Internet of Things
LSTM	Long Short-Term Memory
ML	Machine Learning
ROC	Receiver Operating Characteristic
TTA	Test Time Augmentation
UML	Unified Modeling Language
YOLOv8	You Only Look Once version 8

Symbols and Notations

I	Input image frame from dashboard camera
$F(I)$	Fog density classification output for frame I
C	ConvNeXt-Base feature extractor
V	Estimated visibility in metres
E_{fog}	Fog evidence score from vision correction engine
C_{val}	Contrast value of image region
E_{den}	Edge density of image
S_{haze}	Sky haze estimation score
DCP	Dark channel prior value
$B(I)$	Bounding box prediction from YOLOv8
conf	Confidence score of object detection
d	Proximity distance category (Very close/close/Medium/Far)
α	Learning rate for ConvNeXt-Base fine-tuning
N	Total number of video frames processed
T	Processing time per frame (seconds)

CHAPTER 1

INTRODUCTION

Road safety is one of the most critical concerns for human life, particularly during adverse weather conditions. Fog significantly reduces road visibility, impairs a driver's ability to judge distances, and increases the risk of accidents. Every year, thousands of road accidents occur due to low visibility caused by fog, resulting in loss of life and property. Early and accurate detection of fog density is essential for drivers to take timely precautions and reduce accident risks. Traditional fog detection systems rely on hardware-based sensors such as visibility meters, humidity sensors, and infrared transmissometers. These systems are expensive, require regular calibration and maintenance, and are not widely deployed across all road networks.

Artificial intelligence and deep learning techniques have revolutionized the field of computer vision, enabling automated analysis of images and videos with high accuracy. Convolutional Neural Networks and modern architectures such as ConvNeXt are capable of extracting rich visual features from images for classification tasks. Object detection models such as YOLOv8 can simultaneously identify and localize multiple objects in real time. By combining deep learning-based fog classification with object detection, it is possible to build a comprehensive fog-aware road safety assistance system that works directly from camera input without any specialized hardware.

This project uses a pre-trained ConvNeXt-Base model for fog density classification and YOLOv8 for real-time detection of vehicles, pedestrians, and road signs. A vision-based fog correction engine further refines model predictions using contrast analysis, edge density, sky haze estimation, and the dark channel prior technique. The system processes dashboard camera video frame by frame and overlays a real-time Heads-Up Display with fog level, estimated visibility, speed recommendations, and driver safety alerts. The proposed system provides fast and accurate fog detection, reduces manual effort, and supports smart transportation and advanced driver assistance applications.

1.1 Objective

The main objective of this project is to develop a fog detection and road safety assistance system using hybrid deep learning.

Specific objectives:

- Detect fog conditions automatically from road camera images and video frames
- Classify fog density into four levels: No Fog, Light Fog, Medium Fog, and Dense Fog
- Estimate road visibility in metres based on fog density classification
- Detect vehicles, pedestrians, and road signs simultaneously using YOLOv8
- Provide real-time speed recommendations and safety alerts to drivers
- Detect lane marking loss during medium and dense fog conditions
- Develop a real-time video processing pipeline with HUD overlay

1.2 Motivation

- Fog-related road accidents cause thousands of fatalities and injuries every year, especially on highways and in hilly regions, making early fog detection a critical safety need.
- Drivers often misjudge fog severity, particularly during early morning hours or in unfamiliar terrain, which leads to unsafe driving decisions such as maintaining excessive speed or insufficient braking distance.
- Sensor-based fog detection systems such as visibility meters, humidity sensors, and infrared transmissometers are prohibitively expensive and are not available on most road networks.

- With the rapid advancement of deep learning and computer vision, it is now feasible to develop accurate, camera-based fog detection systems that operate in real time on standard hardware without any specialized equipment.
- The growing availability of pre-trained deep learning models such as ConvNeXt-Base and object detection frameworks such as YOLOv8 makes it possible to build a highly accurate and efficient fog classification system with minimal computational cost.
- There is a strong need for a low-cost, scalable, and intelligent fog safety assistance system that can be integrated directly into vehicles or deployed as part of smart road monitoring infrastructure.

1.3 Background

Table 1.3.1 Literature Survey:

S.NO	TITLE	MERITS	DEMERITS
1	Cross-testing methodology for pattern learning and model transfer in rare fog events detection	Handles class imbalance effectively	Uses meteorological data, not images
2	A reliability anomaly detection method based on enhanced GRU Autoencoder for Vehicular Fog Computing services	Handles concept drift effectively; High detection accuracy; Lightweight compared to LSTM	Focuses on fog computing services, not atmospheric fog; Uses time-series data, not images

3	A New Approach for Nighttime Sea Fog Detection Based on VIIRS/DNB Data	Works at night; high spatial resolution	Depends on lunar illumination conditions
4	Change Point Detection For FOGs Scale Factor Test Data Using Pruned Exact Linear Time (PELT) Algorithm	High detection accuracy; Very low time complexity ($O(n)$); Efficient for large-scale industrial data; Does not require labeled data	Focuses on fiber optic gyroscope (FOG) sensor data, not atmospheric fog; Uses time series numerical data instead of images; Not suitable for camera-based road fog detection
5	Prior Waveform Guided Network (PWGN) for Laser Detection in Fog	High detection accuracy in dense fog; Effectively suppresses fog backscattering noise; Improves ranging accuracy and detection range; CNN-based approach outperforms traditional signal processing methods	Uses LiDAR laser signals instead of camera images; Requires specialized hardware; High computational complexity; Not suitable for low-cost camera based road fog detection
6	Fog-Aware Hierarchical Autoencoder with Density-Based Clustering for AI Driven Threat Detection in Smart Farming IoT Systems	Very high detection accuracy >98%; Unsupervised approach (no labeled data needed); Fog computing friendly and suitable for resource-constrained devices; Noise-resilient clustering using latent-gated DBSCAN; Good generalization across datasets	Designed for IoT cyber-security, not atmospheric road fog detection; Uses network flow data instead of images; Complex architecture with multiple stages; Not directly applicable to vision-based road safety systems

7	AdaFogNet: An Adaptive Preprocessing Based Deep Learning Model for Multi-Class Fog Severity Classification	High accuracy and precision	High computational cost due to VGG16
8	Rain-fog detection via spatial adaptive deformable network with multi-scale feature preservation and task-aware dynamic calibration	High detection accuracy in rain and fog conditions; Handles geometric distortion using deformable convolutions; Preserves multi-scale features for small object detection; Improves real-time performance with higher FPS; Strong generalization across datasets	Complex architecture with high design complexity; Requires high computational resources (GPU); Focused on object detection rather than direct fog density estimation; Not suitable for low-cost or simple camera based fog warning systems
9.	Fog-empowered anomaly detection in IoT networks using one-class asymmetric stacked autoencoder	Very high detection accuracy Unsupervised learning avoids need for labeled data; Efficient dimensionality reduction using OC-ASAE; Suitable for fog/edge environments with low latency; Low false positive rate	Focuses on IoT network security, not atmospheric fog; Uses network traffic data instead of images; Complex architecture increases implementation difficulty; Not applicable for camera-based road fog detection
10	Securing Fog enabled IoT: federated learning and generative	High detection accuracy. Privacy preserving learning. Low latency in fog environments	Not related to road or weather fog. Not image-based. High model complexity

	adversarial networks for intrusion detection		
11	Fog Detection Using Hybrid Deep Learning Models	Combines CNN and LSTM to detect fog more accurately. Uses ResNet-50 for feature extraction and improves fog classification performance.	Requires large datasets and high computational resources. Implementation cost is high due to complex architecture.
12	Machine Learning Based Fog Detection (Patil et al., 2022, Computers and Electronics in Agriculture)	Better than traditional methods; Uses image features	Requires manual feature extraction; Lower accuracy than DL
13	YOLOv8 for Real-Time Object Detection in Adverse Weather (Ultralytics, 2023)	Fast and accurate; Detects multiple objects	Performance drops in extreme fog conditions
14	ConvNeXt: A ConvNet for the 2020s (Liu et al., 2022, CVPR)	Outperforms transformers on many tasks; Strong ImageNet features	Large model size; Requires fine-tuning
15	EfficientNet: Rethinking Model Scaling for CNNs (Tan & Le, 2019, ICML)	Efficient scaling; High accuracy with fewer parameters	Misses global context without transformer

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Project Scope

This project focuses on developing an automated fog detection and road safety assistance system using a deep learning model. The system analyzes road camera video frames and predicts fog density levels using a ConvNeXt-Base classification model. A vision-based correction engine further refines the predictions using computer vision analysis. YOLOv8 object detection is integrated to simultaneously detect vehicles, pedestrians, and road signs. The proposed system helps drivers by providing real-time fog level information, estimated visibility, speed recommendations, brake alerts, and lane loss warnings through a Heads-Up Display overlay. The solution is cost-effective, camera-based, and can be deployed on standard hardware. The system supports real-time video processing and helps improve road safety during foggy conditions.

2.2 Application Areas

- **Highway Safety:** Helps drivers detect fog density in real time on highways and expressways
- **Smart Transportation:** Can be integrated with intelligent transportation systems for automated road monitoring
- **Vehicle ADAS:** Can be integrated with advanced driver assistance systems in modern vehicles
- **Traffic Management:** Supports traffic control centers in issuing fog warnings and speed advisories
- **Research Applications:** Useful for computer vision and road safety research
- **Embedded Systems:** Can be deployed on Raspberry Pi or NVIDIA Jetson for in-vehicle use

- Fleet Management: Useful for monitoring fog safety in commercial vehicle fleets

2.3 Goals

The main aim of this project is to develop an accurate fog detection and road safety assistance system using deep learning.

Accurate Fog Detection

- Develop a ConvNeXt-Base deep learning model for fog density classification
- Classify fog into four levels: No Fog, Light Fog, Medium Fog, Dense Fog

Improved Accuracy

- Use vision-based correction engine for better fog estimation
- Apply contrast, edge density, sky haze, and dark channel prior analysis
- Combine model prediction with physics-based correction for reliability

Real-Time Object Detection

- Integrate YOLOv8 for detecting vehicles, pedestrians, and road signs
- Provide proximity-based alerts (Very Close, Close, Medium, Far)

Driver Safety Guidance

- Provide speed recommendations based on fog density
- Trigger brake alerts for close-proximity objects
- Detect lane marking loss and issue warnings

Scalability

- Support real-time processing of dashboard camera video
- Deployable on standard CPU hardware

2.4 Expected Outcomes

The proposed fog detection and road safety assistance system is expected to provide the following outcomes:

- **Accurate Fog Classification:** The system will classify fog density from road camera images with high accuracy
- **Real-Time Visibility Estimation:** The system will estimate road visibility in metres for each processed frame
- **Object Detection Alerts:** YOLOv8 will detect vehicles and pedestrians and issue proximity-based warnings
- **Speed Recommendations:** Safe driving speeds will be provided based on fog density level
- **Brake Alerts:** Full-screen brake warnings will be triggered when objects are very close
- **Lane Loss Warning:** The system will detect and warn when lane markings are not visible
- **Day/Night Adaptive Guidance:** Safety messages will adapt based on ambient lighting conditions
- **Improved Road Safety:** Early fog detection reduces accident risk and helps drivers take preventive action.

CHAPTER 3

TECHNICAL SPECIFICATIONS

This chapter defines the system requirements and specifications for implementing the fog detection and road safety assistance system. The system is designed to analyze road camera video frames and classify fog density using a ConvNeXt-Base deep learning model. It integrates YOLOv8 for object detection and a vision-based correction engine for improved accuracy. The system provides real-time safety guidance through a HUD overlay. Python, PyTorch, TensorFlow, OpenCV, and Ultralytics YOLOv8 are used for model training and deployment.

3.1 System Requirement

- Video frame input processing capability
- Accurate fog density classification into four levels
- Real-time object detection for vehicles and pedestrians
- Vision-based fog correction engine
- Low prediction latency for real-time processing
- Reliable performance across different weather and lighting conditions
- Support for day and night mode detection.

3.1.1 Hardware Specifications

The proposed system requires basic hardware components to ensure efficient processing and real-time performance. A suitable processor, sufficient memory, and storage are necessary for running the model, while a GPU can improve speed. Input and display devices are used for capturing and visualizing results.

Table 3.1 Hardware Specification

Component	Specification
Processing Unit	Intel Core i5 / Ryzen 5 or above (GPU recommended)
RAM	Minimum 8 GB
Storage	Minimum 10 GB free space
Input Device	Dashboard Camera / Laptop / Desktop
Display	Monitor or Laptop screen
GPU (Optional)	NVIDIA GPU for faster inference
Internet	Required for dataset download and model weights

3.1.2 Software Specifications

Table 3.2 Software Specification

Software	Version/Tools used
Operating System	Windows 10 / Linux / macOS
Programming Language	Python 3.10+
Deep Learning Framework	PyTorch, TensorFlow / Keras
Fog Classification Model	ConvNeXt-Base (fb_in22k_ft_in1k) via timm
Object Detection Model	YOLOv8n (Ultralytics)

Computer Vision Library	OpenCV (cv2)
Image Processing	PIL / Pillow, NumPy
Data Augmentation	torchvision.transforms, Albumentations
Visualization	Matplotlib, Seaborn
Dataset	Foggy Cityscapes (via KaggleHub)
Development Environment	Jupyter Notebook / VS Code
Web Framework (optional)	Flask

3.2 Justification of Specifications

- Processing Unit: Intel i5 or higher ensures efficient model inference and video processing in real time.
- RAM & Storage: Minimum 8 GB RAM supports deep learning model loading. Sufficient storage is needed for the dataset, model weights, and output videos.
- GPU Support: A GPU significantly improves frame processing speed and reduces inference time for YOLOv8 and ConvNeXt models.
- Software Tools: Python and PyTorch provide flexible deep learning implementation. OpenCV handles real-time video frame capture and annotation.
- Libraries: timm library supports ConvNeXt-Base architecture. Albumentations and torchvision provide augmentation support. Ultralytics provides YOLOv8 integration.
- Development Environment: Jupyter Notebook enables interactive model development and result visualization.

CHAPTER 4

DESIGN APPROACH AND DETAILS

This chapter describes the design approach and methodology used for developing the fog detection and road safety assistance system. The system is designed to classify fog density from road video frames and provide real-time driver guidance using ConvNeXt-Base and YOLOv8 architectures. The proposed system includes image preprocessing, fog classification, vision-based correction, object detection, guidance generation, and HUD rendering. The objective is to build an accurate, efficient, and real-time road safety system.

4.1 Design Approach

The image illustrates the complete pipeline of a Fog Detection and Road Safety Assistance System. The process begins with the Foggy Cityscapes Dataset, which contains real road images classified into three fog conditions — No Fog, Medium Fog, and Dense Fog. This dataset is divided into three subsets: 70% for training, 15% for validation, and 15% for testing, ensuring the model learns effectively and is evaluated on unseen data.

Before the images are passed into the deep learning model, they undergo a comprehensive Preprocessing and Augmentation pipeline to improve data quality and model generalization. Each image is resized to a standard dimension of 224×224 pixels and normalized using ImageNet mean and standard deviation values to ensure consistent input distribution. A batch size of 32 is used during training, and the model is trained over 10 to 15 epochs with L2 regularization (weight decay of $1e-4$) and optional L1 regularization to prevent overfitting. Data augmentation techniques such as horizontal flip, rotation, color jitter, and Gaussian blur are applied to artificially increase dataset diversity and make the model more robust to real-world variations in lighting, angle, and weather. The core classification model is a pre-trained ConvNeXt-Base network, a modern and powerful convolutional architecture that is fine-tuned for fog density classification using CrossEntropy loss and the Adam optimizer. The model's effectiveness is thoroughly evaluated using multiple performance metrics including Accuracy, Precision, Recall, F1-Score, Confusion

Matrix, and the ROC Curve with AUC score, providing a complete and holistic picture of how well the model performs across all fog categories.

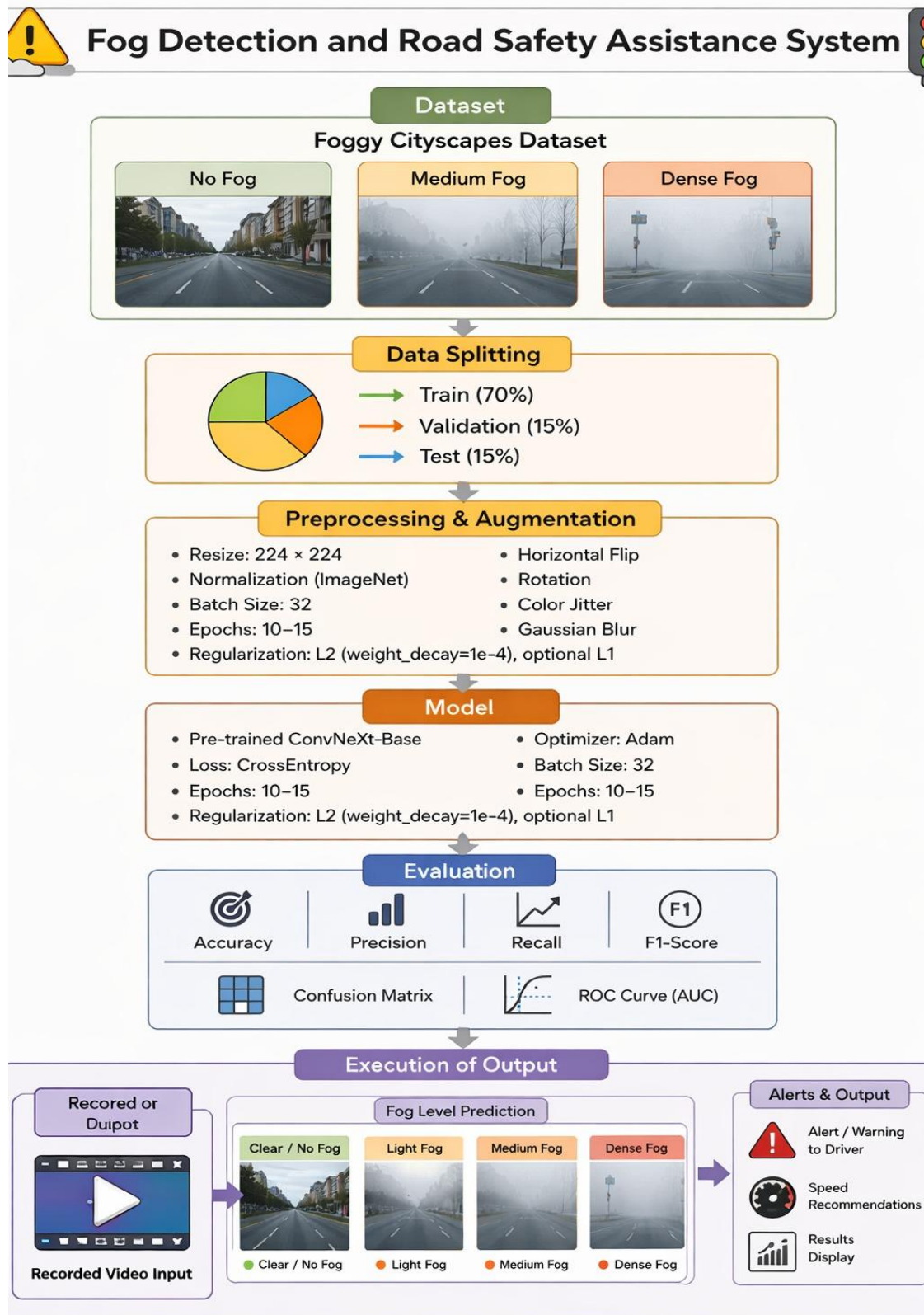


Fig. 4.1.1 System Architecture

In the final and most impactful stage — Execution of Output — the trained model is deployed on a live real-time dashboard camera video feed to simulate actual driving conditions. Each video frame is captured and instantly classified into one of four fog severity levels: Clear/No Fog, Light Fog, Medium Fog, or Dense Fog, with the predictions displayed visually on screen in an intuitive and color-coded format. Based on the fog level detected in each frame, the system automatically generates intelligent driver alerts and safety outputs in real time. These outputs include warning alerts to notify the driver of dangerous fog ahead, speed recommendations to guide the driver to reduce speed appropriately for the current visibility conditions, and a results display showing the ongoing fog analysis. This makes the system not just a classification tool but a complete intelligent road safety assistant that operates seamlessly in real time, without the need for any expensive hardware sensors, making it a highly practical, scalable, and cost-effective solution for improving road safety during foggy and low-visibility driving conditions.

UML Diagram:

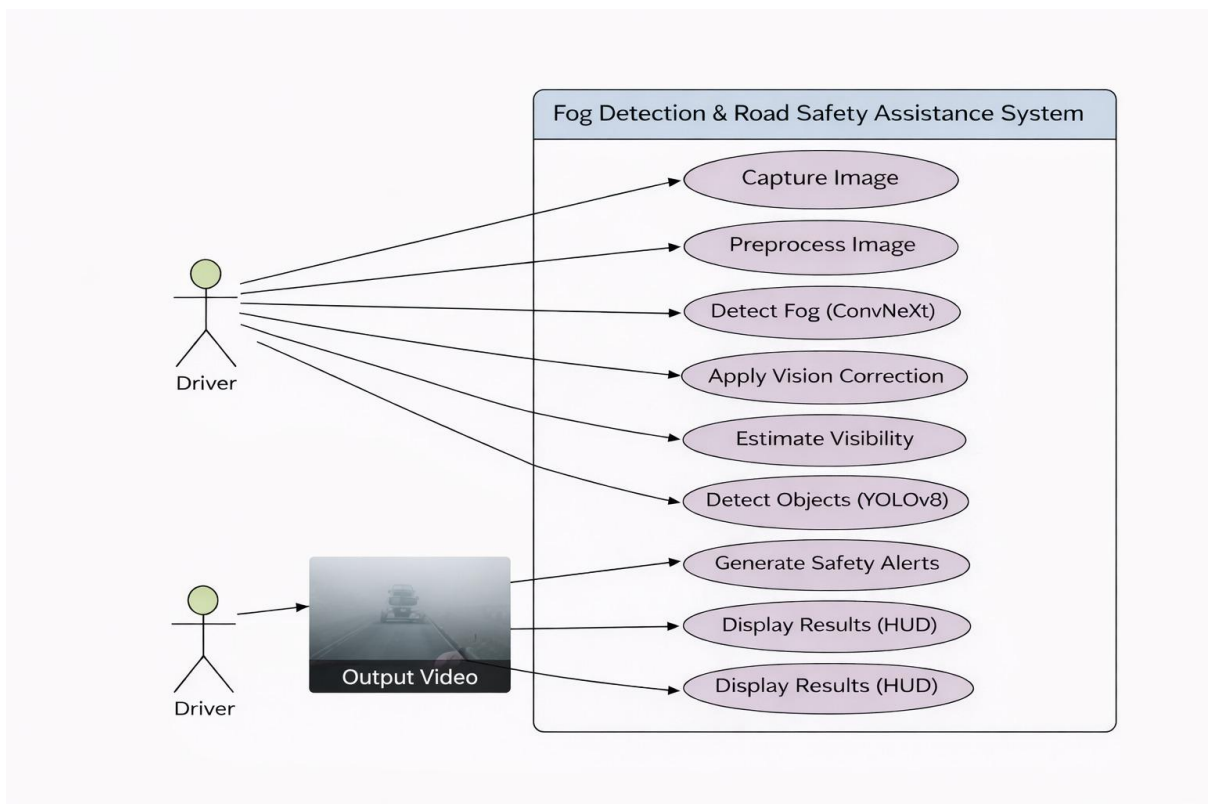


Fig. 4.1.2 UML Diagram

This use case diagram represents the Fog Detection & Road Safety Assistance System, showing how the Driver actor interacts with the system through multiple use cases. The Driver connects to six core system functions — Capture Image, Preprocess Image, Detect Fog using ConvNeXt, Apply Vision Correction, Estimate Visibility, and Detect Objects using YOLOv8 — all of which form the main processing pipeline. A second Driver actor at the bottom represents the output side, interacting via the Output Video which feeds into Generate Safety Alerts, Display Results via HUD, and Save Output Video. The use cases are represented as pink-shaded ovals inside the system boundary rectangle, following standard UML notation where the boundary separates internal system functions from external actors. Overall the diagram clearly illustrates that the Driver is both the initiator of the system by providing dashcam video input and the receiver of the system's output in the form of real-time HUD alerts and safety guidance.

4.2 Codes and Standards

The system is developed using standard coding practices to ensure reliability and maintainability. Python programming language is used for implementing deep learning models and video processing pipeline. The code is modularized into preprocessing, model loading, fog classification, vision correction, object detection, guidance generation, and HUD rendering components.

Coding Standards

- Python code follows PEP8 guidelines
- Modular code structure is used with separate functions for each pipeline stage
- Meaningful variable names are used throughout
- Comments added for all major functions and processing blocks
- Exception handling implemented for video reading and model loading

Software Standards

- PyTorch and TensorFlow used for model training and inference
- timm library used for ConvNeXt-Base architecture
- Ultralytics used for YOLOv8 object detection
- OpenCV used for video processing and HUD rendering
- Albumentations and torchvision used for data augmentation

4.3 Constraints, Alternatives and Tradeoffs

Constraints:

- The system requires a GPU for real-time processing at acceptable frame rates (15+ FPS); CPU-only processing is slower
- Fog classification accuracy depends on training dataset quality; night-time or rain scenarios may reduce accuracy
- YOLOv8 detection performance decreases in extreme dense fog conditions where objects are visually indistinguishable
- The vision correction engine uses fixed thresholds which may need calibration for different geographic regions and seasons
- Model retraining is required to support additional fog categories or different camera perspectives

Alternatives Considered:

- EfficientNet-B3 and ResNet-50 were tested but achieved only 30-60% accuracy due to poor texture modeling in foggy conditions
- LiDAR-based fog detection was considered but rejected due to high hardware cost and unavailability in standard vehicles
- LSTM-based temporal models were evaluated for sequential frame analysis but added complexity without significant accuracy gain

Tradeoffs:

- Accuracy vs. Speed: ConvNeXt-Base is more accurate than EfficientNet but has higher inference latency per frame
- Safety vs. Confidence: The vision correction engine trades model confidence for conservative (safer) predictions
- Generalization vs. Specialization: Heavy augmentation improves generalization but may slightly reduce accuracy on the specific training distribution

Table 4.3.1 Tradeoff and Description

Trade-off	Description
Accuracy vs Speed	ConvNeXt-Base provides higher accuracy but slower inference than lightweight models
Complexity vs Performance	Vision correction engine improves reliability but adds processing overhead
Dataset Size vs Training Time	Larger foggy dataset improves accuracy but increases training duration
CPU vs GPU Deployment	GPU provides real-time performance; CPU is accessible but slower

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

5.1 Project Timeline

The project timeline shows the sequential development of the fog detection and road safety assistance system. The project is divided into seven tasks. Each task is scheduled with specific duration and milestones to ensure proper project completion.

Phase	Activity	Duration	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8	Wk 9	Wk 10	Wk 11	Wk 12
Phase I	Planning & Requirement Analysis Define objectives, scope, hardware & software specs	4 days												
	Dataset Collection Download Foggy Cityscapes via KaggleHub · verify class balance	5 days												
Phase II	Data Preprocessing & Augmentation Resize · normalize · CLAHE · flip · rotate · color jitter	6 days												
	ConvNeXt-Base Model Training Fine-tune on No Fog · Medium Fog · Dense Fog · AdamW · cosine LR	21 days												
Phase III	Model Evaluation ROC AUC · confusion matrix · TTA · target accuracy 90%+	included												
	Vision Correction Engine Contrast · edge density · sky haze · dark channel prior	14 days												
Phase IV	YOLOv8 Integration Detect vehicles · pedestrians · road signs · proximity alerts	4 days												
	Real-Time Pipeline & HUD End-to-end video pipeline · fog banner · speed badge · brake alert · lane warning	5 days												
Phase V	Testing & Documentation Test on 1,624-frame dashcam video · validate analytics · complete report	7 days												

Fig. 5.1 Project Timeline

Phase I — Planning & Requirement Analysis

- Completed entirely in Week 1.
- Covers defining objectives, project scope, and hardware & software specifications.
- Shown in purple color.

Phase II — Dataset Collection & Preprocessing

- Dataset Collection (5 days) runs across Week 1 and Week 2, shown in cyan blue.
- Data Preprocessing & Augmentation (6 days) runs across Week 2 and Week 3, shown in teal green.
- Tasks include resizing, normalization, CLAHE enhancement, flipping, rotation, and color jitter augmentation.

Phase III — Model Training & Evaluation

- ConvNeXt-Base Model Training runs from Week 3 to Week 6, shown in bright green, taking 21 days.
- Model Evaluation is included within the same phase and runs across Week 6 and Week 7, shown in light green.
- Evaluation includes ROC AUC, confusion matrix, Test Time Augmentation, targeting 90%+ accuracy.

Phase IV — Vision Correction Engine & YOLOv8 Integration

- Vision Correction Engine (14 days) runs from Week 7 to Week 9, shown in orange.
- Covers contrast analysis, edge density, sky haze estimation, and dark channel prior technique.
- YOLOv8 Integration (4 days) runs across Week 9 and Week 10, shown in yellow-orange.
- Detects vehicles, pedestrians, road signs with proximity alert logic.

Phase V — Pipeline, HUD & Documentation

- Real-Time Pipeline & HUD (5 days) runs across Week 10 and Week 11, shown in pink/magenta.
- Builds the end-to-end video pipeline with fog banner, speed badge, brake alert, and lane warning overlay.
- Testing & Documentation (7 days) runs across Week 11 and Week 12, shown in blue.
- Tests on 1,624 dashcam video frames, validates fog analytics, and completes the final project report.

5.2 Project Timeline Summary

The project schedule describes the sequential development of the fog detection and road safety assistance system. The project starts with defining system requirements and collecting the Foggy Cityscapes dataset via KaggleHub. The dataset is preprocessed using resizing, normalization, and augmentation techniques. The ConvNeXt-Base fog classification model is trained and fine-tuned on the preprocessed dataset. The vision-based fog correction engine is developed and integrated. YOLOv8 is integrated for real-time object detection. The complete video processing pipeline with HUD overlay is assembled and tested. Finally, the results and documentation are prepared. Each task is scheduled with proper duration to ensure timely completion.

Table 5.2.1 Milestone and Description

S.No	Milestone	Description
1	System Requirements Finalized	Defined hardware, software, and dataset requirements
2	Dataset Collected	Downloaded Foggy Cityscapes dataset via KaggleHub
3	Data Preprocessing Completed	Resized, normalized, and augmented dataset
4	ConvNeXt Model Trained	Model trained with 90%+ accuracy on test set
5	Vision Correction Engine Ready	Contrast, edge, sky haze, dark channel prior analysis
6	YOLOv8 Integration Completed	Object detection pipeline validated
7	Full Pipeline and Documentation	Real-time video pipeline deployed and report completed

5.3 Conclusion

The structured project schedule ensures systematic development of the fog detection and road safety assistance system. Each milestone contributes to the successful implementation of the deep learning model and real-time pipeline. Following the planned timeline helps achieve accurate fog detection and efficient video deployment. The final system provides a reliable and scalable solution for fog detection and driver safety guidance in real-world road conditions.

CHAPTER 6

PROJECT DEMONSTRATION

6.1 Sample Codes

6.1.1 Deep Learning Code

```
import torch

import torch.nn as nn

import torch.optim as optim

from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler

import torchvision.transforms as transforms

import timm

import albumentations as A

from albumentations.pytorch import ToTensorV2

import random

import time

from sklearn.utils.class_weight import compute_class_weight

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

print(f'Device: {DEVICE}')

if torch.cuda.is_available():

    print(f'GPU: {torch.cuda.get_device_name(0)}')
```

```

print(f'VRAM: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB')

CLASSES = sorted(df_preprocessed['label'].unique().tolist())

CLASS2IDX = {c: i for i, c in enumerate(CLASSES)}

IDX2CLASS = {i: c for c, i in CLASS2IDX.items()}

NUM_CLASSES = len(CLASSES)

print(f'Classes: {CLASS2IDX}')

BATCH_SIZE = 32

NUM_WORKERS = 2

train_ds = FogDataset(df_train, transform=get_transforms('train'))

val_ds = FogDataset(df_val, transform=get_transforms('val'))

test_ds = FogDataset(df_test, transform=get_transforms('test'))

train_labels = [CLASS2IDX[l] for l in df_train['label']]

class_counts = Counter(train_labels)

weights = [1.0 / class_counts[l] for l in train_labels]

sampler = WeightedRandomSampler(weights, num_samples=len(weights),
replacement=True)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, sampler=sampler,

                           num_workers=NUM_WORKERS, pin_memory=True)

val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,

                        num_workers=NUM_WORKERS, pin_memory=True)

test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False,

```

```

num_workers=NUM_WORKERS, pin_memory=True)

print(f'Train  batches:  {len(train_loader)}  |  Val:  {len(val_loader)}  |  Test:
{len(test_loader)}')

cw = compute_class_weight('balanced', classes=np.array(range(NUM_CLASSES)),
y=train_labels)

class_weights = torch.FloatTensor(cw).to(DEVICE)

print(f'Class weights: {dict(zip(CLASSES, cw.round(3)))}')

class FogClassifier(nn.Module):

    def __init__(self, num_classes=3, dropout=0.3):

        super().__init__()

        # Load ConvNeXt-Base pretrained on ImageNet-22k → fine-tuned on 1k

        self.backbone = timm.create_model(

            'convnext_base.fb_in22k_ft_in1k', # Best pretrained weights

            pretrained=True,

            num_classes=0, # Remove classifier

            global_pool='avg'

        )

        feat_dim = self.backbone.num_features # 1024 for convnext_base

        self.classifier = nn.Sequential(

            nn.Linear(feat_dim, 512),

            nn.LayerNorm(512),

            nn.GELU(),

```

```

        nn.Dropout(dropout),

        nn.Linear(512, 128),

        nn.LayerNorm(128),

        nn.GELU(),

        nn.Dropout(dropout / 2),

        nn.Linear(128, num_classes)

    )

    def forward(self, x):

        features = self.backbone(x)

        return self.classifier(features)

model = FogClassifier(num_classes=NUM_CLASSES, dropout=0.3).to(DEVICE)

total_params = sum(p.numel() for p in model.parameters())

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'Total params: {total_params/1e6:.1f}M')

print(f'Trainable params: {trainable_params/1e6:.1f}M')

print('✅ ConvNeXt-Base model built')


EPOCHS = 40

LR_HEAD = 1e-3    # Higher LR for new head

LR_BACKBONE = 5e-5 # Lower LR for pretrained backbone (key!)

WEIGHT_DECAY = 0.01

```

```

MIXUP_ALPHA = 0.4

LABEL_SMOOTHING = 0.1

PATIENCE = 8 # Early stopping

optimizer = optim.AdamW([

    {'params': model.backbone.parameters(), 'lr': LR_BACKBONE},

    {'params': model.classifier.parameters(), 'lr': LR_HEAD}

], weight_decay=WEIGHT_DECAY)

criterion = nn.CrossEntropyLoss(

    weight=class_weights,

    label_smoothing=LABEL_SMOOTHING

)

scheduler = optim.lr_scheduler.OneCycleLR(

    optimizer,

    max_lr=[LR_BACKBONE, LR_HEAD],

    steps_per_epoch=len(train_loader),

    epochs=EPOCHS,

    pct_start=0.1, # 10% warmup

    anneal_strategy='cos'

)

scaler = torch.cuda.amp.GradScaler(enabled=torch.cuda.is_available())

print('✅ Training setup complete')

```

```

def train_epoch(model, loader, optimizer, criterion, scheduler, scaler,
use_mixup=True):

    model.train()

    total_loss, total_correct, total = 0, 0, 0

    for imgs, labels in loader:

        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)

        if use_mixup and random.random() > 0.5:

            imgs, labels_a, labels_b, lam = mixup_data(imgs, labels, MIXUP_ALPHA)

            with torch.cuda.amp.autocast(enabled=torch.cuda.is_available()):

                outputs = model(imgs)

                loss = mixup_criterion(criterion, outputs, labels_a, labels_b, lam)

                preds = outputs.argmax(dim=1)

                correct = (lam * (preds == labels_a).float() + (1-lam) * (preds ==
labels_b).float()).sum()

            else:

                with torch.cuda.amp.autocast(enabled=torch.cuda.is_available()):

                    outputs = model(imgs)

                    loss = criterion(outputs, labels)

                    preds = outputs.argmax(dim=1)

                    correct = (preds == labels).sum().item()

```



```

optimizer.zero_grad()

scaler.scale(loss).backward()

scaler.unscale_(optimizer)

torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

scaler.step(optimizer)

scaler.update()

scheduler.step()


total_loss += loss.item() * imgs.size(0)

total_correct += correct if isinstance(correct, (int, float)) else correct.item()

total += imgs.size(0)


return total_loss / total, total_correct / total


@torch.no_grad()

def eval_epoch(model, loader, criterion):

    model.eval()

    total_loss, total_correct, total = 0, 0, 0

    all_preds, all_labels = [], []


    for imgs, labels in loader:

```

```

imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)

with torch.cuda.amp.autocast(enabled=torch.cuda.is_available()):

    outputs = model(imgs)

    loss = criterion(outputs, labels)

    preds = outputs.argmax(dim=1)

    total_loss += loss.item() * imgs.size(0)

    total_correct += (preds == labels).sum().item()

    total += imgs.size(0)

    all_preds.extend(preds.cpu().numpy())

    all_labels.extend(labels.cpu().numpy())

return total_loss / total, total_correct / total, all_preds, all_labels

print('✅ Training loop ready')

history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

best_val_acc = 0.0

patience_counter = 0

MODEL_PATH = '/content/best_fog_model.pt'

print('='*65)

```

```

print(f{"Epoch":>6} | {"Train Loss":>10} | {"Train Acc":>9} | {"Val Loss":>8} |
{"Val Acc":>8} | {"LR":>8}')

print('='*65)

```

```

for epoch in range(1, EPOCHS + 1):

```

```

    t0 = time.time()

```

```

        train_loss, train_acc = train_epoch(model, train_loader, optimizer, criterion,
scheduler, scaler)

```

```

        val_loss, val_acc, _, _ = eval_epoch(model, val_loader, criterion)

```

```

        elapsed = time.time() - t0

```

```

        history['train_loss'].append(train_loss)

```

```

        history['val_loss'].append(val_loss)

```

```

        history['train_acc'].append(train_acc)

```

```

        history['val_acc'].append(val_acc)

```

```

        current_lr = optimizer.param_groups[1]['lr'] # Head LR

```

```

        print(f{"epoch":>6} | {"train_loss":>10.4f} | {"train_acc":>8.2%} | {"val_loss":>8.4f} |
{"val_acc":>8.2%} | {"current_lr:.2e}', end="")

```

```

        if val_acc > best_val_acc:

```

```

            best_val_acc = val_acc

```

```

            patience_counter = 0

```

```

torch.save({

    'epoch': epoch,

    'model_state_dict': model.state_dict(),

    'optimizer_state_dict': optimizer.state_dict(),

    'val_acc': val_acc,

    'classes': CLASSES,

    'class2idx': CLASS2IDX,

}, MODEL_PATH)

print(' ← 🏆 Best!')

else:

    patience_counter += 1

    print(f' (patience: {patience_counter}/{PATIENCE})')

    if patience_counter >= PATIENCE:

        print(f'\nEarly stopping at epoch {epoch}')

        break

print('\n' + '='*65)

print(f' 🏆 Best Validation Accuracy: {best_val_acc:.2%}')

```

6.1.2 Video Processing Code

```
import os

import cv2

import torch

import torch.nn as nn

import numpy as np

from PIL import Image

from torchvision.models import efficientnet_v2_s, EfficientNet_V2_S_Weights

import torchvision.transforms as T

from ultralytics import YOLO

import time

import warnings

import matplotlib.pyplot as plt

import matplotlib.patches as mpatches

from collections import deque

warnings.filterwarnings('ignore')


DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

print(f 🟢 Using device: {DEVICE})
```

```

if torch.cuda.is_available():

    print(f' GPU: {torch.cuda.get_device_name(0)}')

def show_sample_frames(video_path, n=6):

    cap_out = cv2.VideoCapture(video_path)

    if not cap_out.isOpened():

        print("Output video not ready yet.")

        return

    total = int(cap_out.get(cv2.CAP_PROP_FRAME_COUNT))

    indices = [int(i * total / n) for i in range(n)]

    fig, axes = plt.subplots(2, 3, figsize=(18, 9))

    fig.patch.set_facecolor('#111111')

    fig.suptitle(' 🚗 Fog-Aware Driving Assistant v2 — Sample Frames',

                 color='white', fontsize=15, fontweight='bold', y=1.01)

    for ax, fi in zip(axes.flat, indices):

        cap_out.set(cv2.CAP_PROP_POS_FRAMES, fi)

        ret, frame = cap_out.read()

        if ret:

            ax.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

            ax.set_title(f'Frame {fi}', color='white', fontsize=10)

            ax.axis('off')

```

```

        ax.set_facecolor('#111111')

plt.tight_layout()

plt.savefig('sample_frames_v2.png',          dpi=100,          bbox_inches='tight',
facecolor='#111111')

plt.show()

cap_out.release()

print("✅ Sample frames saved to: sample_frames_v2.png")

show_sample_frames(OUTPUT_VIDEO)

CLASS_COLOR_MAP = {

    'clear'      : '#32CD32',

    'light_fog'  : '#00C8FF',

    'medium_fog' : '#FFA500',

    'dense_fog'  : '#FF3030',

}

CLASS_NUM = {'clear': 0, 'light_fog': 1, 'medium_fog': 2, 'dense_fog': 3}

# --- Unpack timeline data ---

tl_frames = [f for f, _ in fog_timeline]

tl_vals  = [CLASS_NUM[c] for _, c in fog_timeline]

tl_colors = [CLASS_COLOR_MAP[c] for _, c in fog_timeline]

vis_frames = [f for f, _ in visibility_timeline]

vis_vals  = [v for _, v in visibility_timeline]

```

```

fig, axes = plt.subplots(3, 1, figsize=(16, 12))

fig.patch.set_facecolor('#111111')

fig.suptitle(' 🚗 Fog-Aware Driving Assistant v2 — Analytics',

             color='white', fontsize=15, fontweight='bold')

ax = axes[0]

ax.set_facecolor('#1a1a1a')

ax.scatter(tl_frames, tl_vals, c=tl_colors, s=4, alpha=0.7)

ax.set_yticks([0, 1, 2, 3])

ax.set_yticklabels(['Clear', 'Light Fog', 'Medium Fog', 'Dense Fog'], color='white')

ax.set_xlabel('Frame', color='#aaaaaa')

ax.set_title('Fog Level Timeline (with Correction)', color='white', fontsize=12)

ax.tick_params(colors='white')

ax.spines[:].set_color('#444444')

patches = [mpatches.Patch(color=v, label=k.replace('_', ' ').title()) for k,v in
CLASS_COLOR_MAP.items()]

ax.legend(handles=patches, facecolor='#1a1a1a', labelcolor='white', framealpha=0.6)

ax2 = axes[1]

ax2.set_facecolor('#1a1a1a')

ax2.plot(vis_frames, vis_vals, color='#00C8FF', linewidth=1.0, alpha=0.8)

ax2.fill_between(vis_frames, vis_vals, alpha=0.2, color='#00C8FF')

ax2.axhline(y=200, color='#FFA500', linestyle='--', linewidth=0.8, label='200m (light
fog threshold)')

```



```

ax2.axhline(y=50, color='#FF3030', linestyle='--', linewidth=0.8, label='50m
(medium fog threshold)')

ax2.set_xlabel('Frame', color='#aaaaaa')

ax2.set_ylabel('Visibility (m)', color='#aaaaaa')

ax2.set_title('Visibility Distance per Frame', color='white', fontsize=12)

ax2.tick_params(colors='white')

ax2.spines[:].set_color('#444444')

ax2.legend(facecolor='#1a1a1a', labelcolor='white', framealpha=0.6)

ax3 = axes[2]

ax3.set_facecolor('#1a1a1a')

labels_bar = [c for c, v in fog_counts.items() if v > 0]

counts_bar = [fog_counts[c] * FOG_PREDICT_EVERY for c in labels_bar]

colors_bar = [CLASS_COLOR_MAP[c] for c in labels_bar]

bars = ax3.bar([l.replace('_', '\n') for l in labels_bar], counts_bar,

               color=colors_bar, edgecolor='white', linewidth=0.8)

ax3.set_title('Approx. Frames per Fog Level', color='white', fontsize=12)

ax3.set_ylabel('Frames (approx)', color='#aaaaaa')

ax3.tick_params(colors='white')

ax3.spines[:].set_color('#444444')

for bar in bars:

    bh = bar.get_height()

    ax3.text(bar.get_x() + bar.get_width() / 2, bh + 1, f'{int(bh)}',

```

```

        ha='center', color='white', fontsize=10)

plt.tight_layout()

plt.savefig('fog_analytics_v2.png',          dpi=100,          bbox_inches='tight',
facecolor='#111111')

plt.show()

print("✅ Analytics chart saved to: fog_analytics_v2.png")

```

6.2 Sample Output:

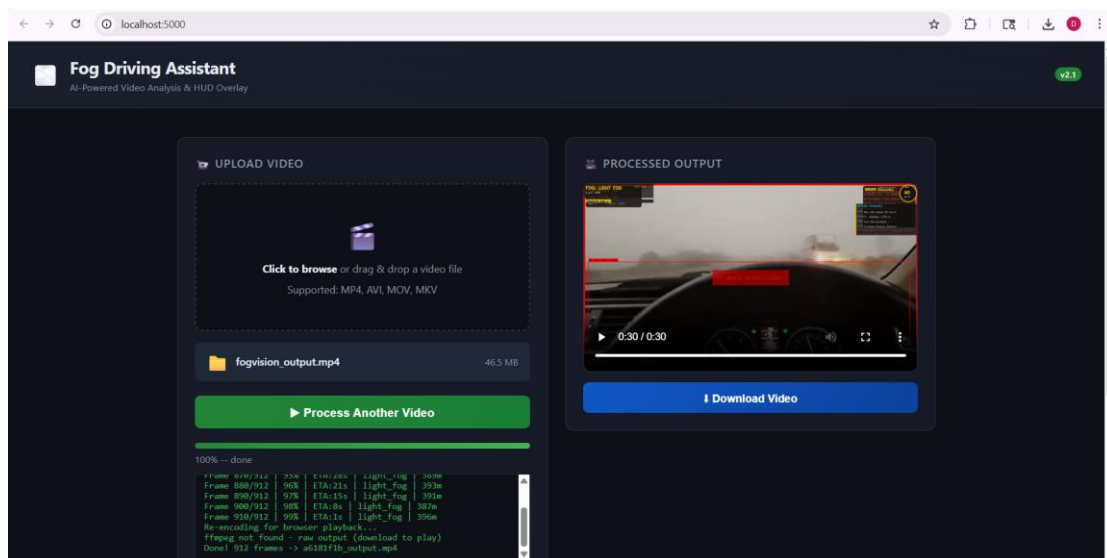


Fig 6.1 Image upload and output video

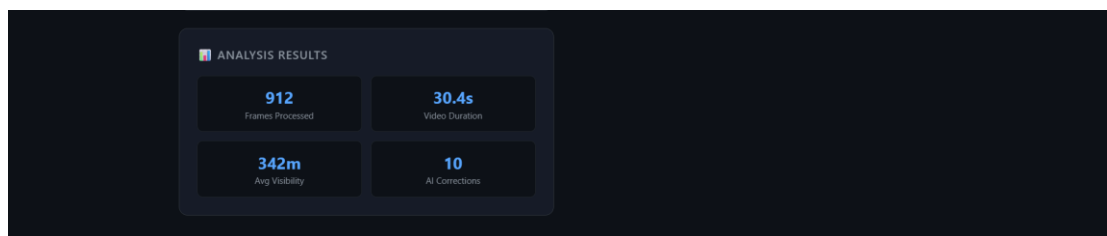


Fig 6.2 Analysis Result

CHAPTER 7

RESULT & DISCUSSION

7.1 Dataset Description

The dataset used in this project is the Foggy Cityscapes dataset, downloaded via the KaggleHub API. It contains road images organized into class folders representing different fog density levels. The dataset includes both foggy and clear road images collected from urban driving scenarios under varying weather and lighting conditions. Each image is labelled according to its fog density category, organized in a folder structure compatible with deep learning frameworks.

The dataset includes three primary fog categories: No Fog, Medium Fog, and Dense Fog, with approximately 500 images per class. For the driving assistant system, an additional Light Fog category is inferred during vision correction. The images show diverse road conditions including highways, city streets, and residential areas, captured under different times of day and weather intensities.

No Fog:



Medium Fog:



Dense Fog:



Fig. 7.1 Dataset Images

Data preprocessing techniques including resizing to 224×224 pixels, normalization using ImageNet statistics, and augmentation techniques such as horizontal flipping, rotation ($\pm 10^\circ$), colour jitter, Gaussian blur, motion blur, random fog, random rain, affine and perspective transforms are applied. The dataset is split into training (70%), validation (15%), and test (15%) sets using stratified sampling to preserve class distribution.

7.2 Intermediate results:

7.2.1 Visualization Images:



Fig 7.2.1 Visualization Image

The image shows road scenes classified into three fog levels: No Fog, Medium Fog, and Dense Fog. Clear visibility is observed in No Fog, while Medium Fog slightly reduces clarity. Dense Fog significantly lowers visibility, making objects hard to detect. This helps in training models to identify fog intensity.

7.2.2 Preprocessing Image:

The image illustrates the preprocessing steps and dataset analysis performed for fog classification. The top section shows road images from three fog categories—No Fog, Medium Fog, and Dense Fog—before and after applying CLAHE (Contrast Limited Adaptive Histogram Equalization) and resizing, which improves image clarity and contrast. The class imbalance chart indicates that the dataset is well-balanced, with an equal ratio of samples across all three classes, ensuring unbiased model training. Additionally, the brightness overlap histogram reveals that the distributions of the fog classes significantly overlap, indicating that the data is not linearly separable and requires advanced models for accurate classification.

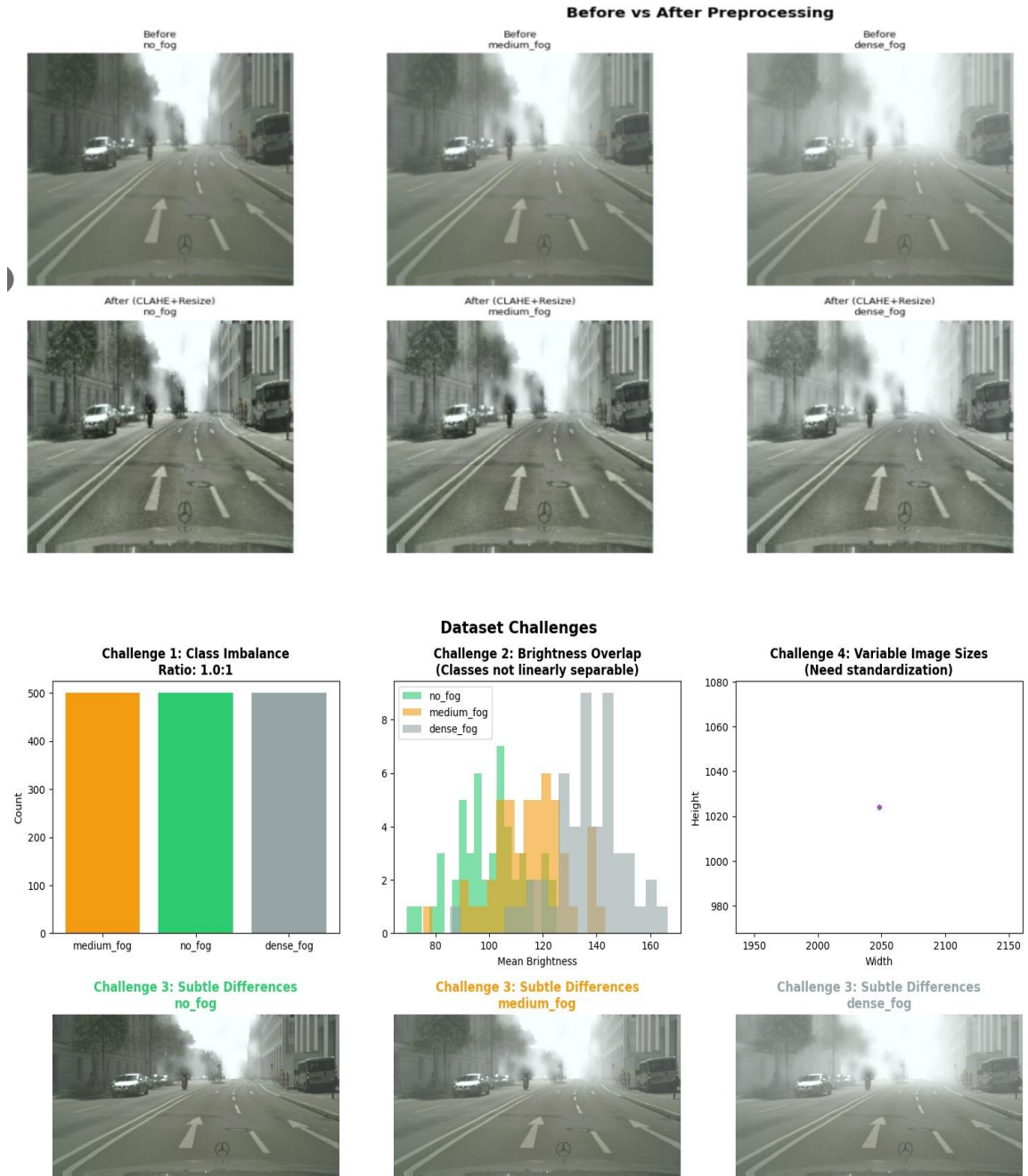


Fig. 7.2.2 Preprocessed Images

Furthermore, the variable image size chart highlights that the original images have different resolutions, emphasizing the need for resizing them to a standard size such as 224×224 pixels for consistent processing. The subtle differences panel demonstrates that the visual differences between No Fog, Medium Fog, and Dense Fog images are minimal to the human eye. This further justifies the use of a powerful deep learning model like ConvNeXt-Base, which can effectively capture fine-grained features and accurately distinguish between different fog density levels.

7.2.3 Class Distribution Bar Chart:

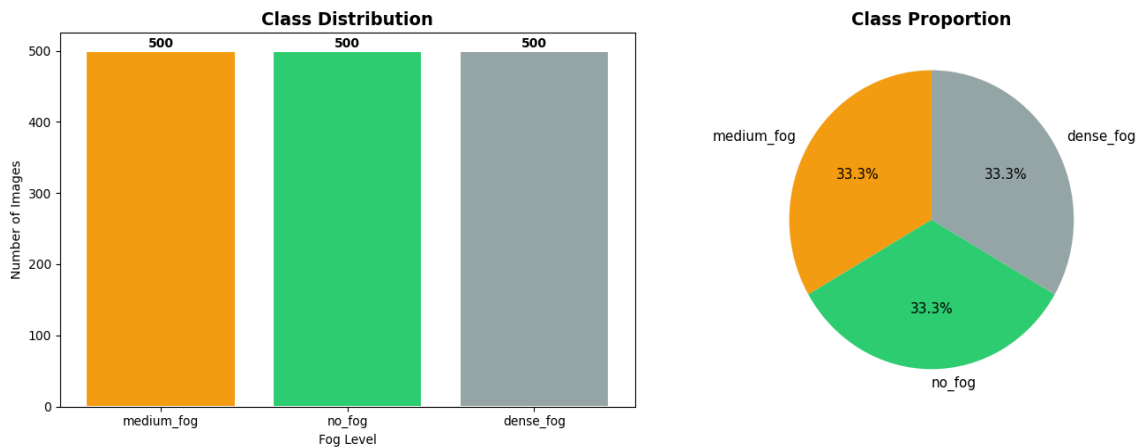
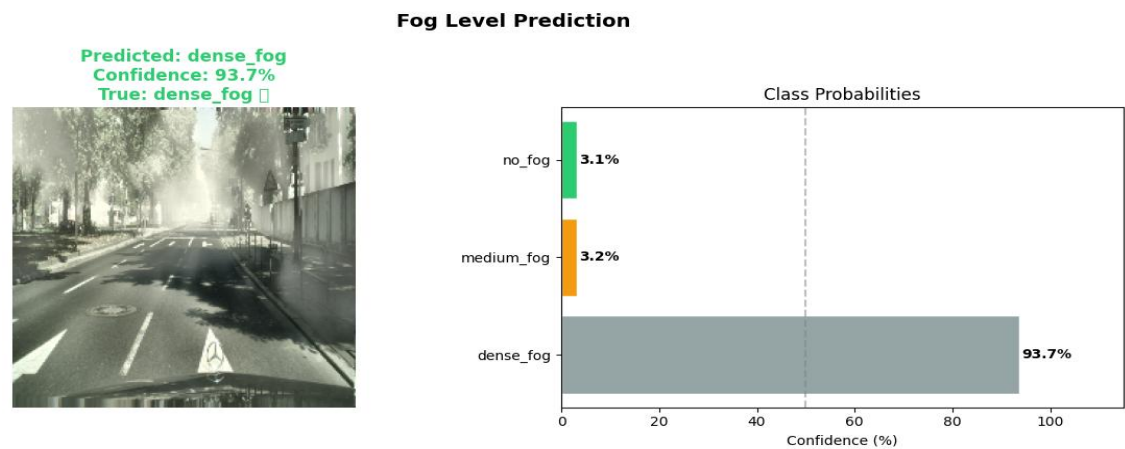


Fig. 7.2.3 Class Distribution Bar Chart

This image shows the Class Distribution and Class Proportion of the Foggy Cityscapes dataset used in your project. The bar chart on the left confirms that each of the three fog categories — medium_fog, no_fog, and dense_fog — contains exactly 500 images, making the dataset perfectly balanced. The pie chart on the right visually reinforces this balance, with each class occupying exactly 33.3% of the total dataset. This equal distribution across all three classes is highly beneficial for training the ConvNeXt-Base model, as it eliminates class imbalance bias and ensures the model learns equally well from all fog density levels. As a result, no additional techniques such as class weighting or oversampling were required during the training process.

7.2.4 Prediction Image:



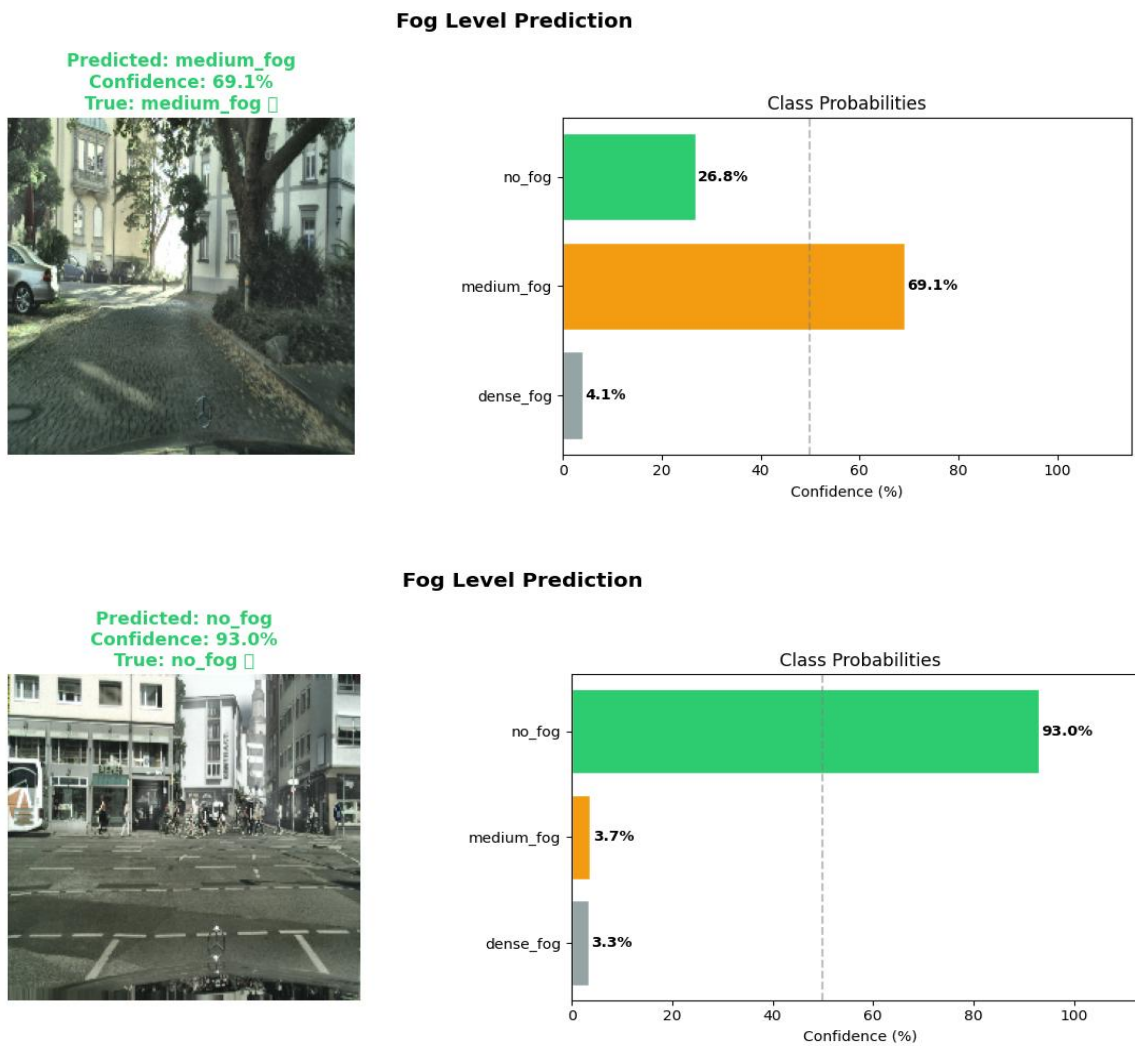


Fig. 7.2.4 Prediction Images

The image presents three fog level predictions from a machine learning model applied to real road scenes, each paired with a probability bar chart for dense fog, medium fog, and no fog classes. The first scene is classified as dense fog with 93.7% confidence, the second as medium fog with 69.1% confidence, and the third as no fog with 93.0% confidence — all predictions matching the true labels. The model shows highest certainty on the extreme classes while medium fog yields lower confidence due to its visual similarity with both other categories, which is consistent with earlier confusion matrix results.

7.3 Final Output Image:

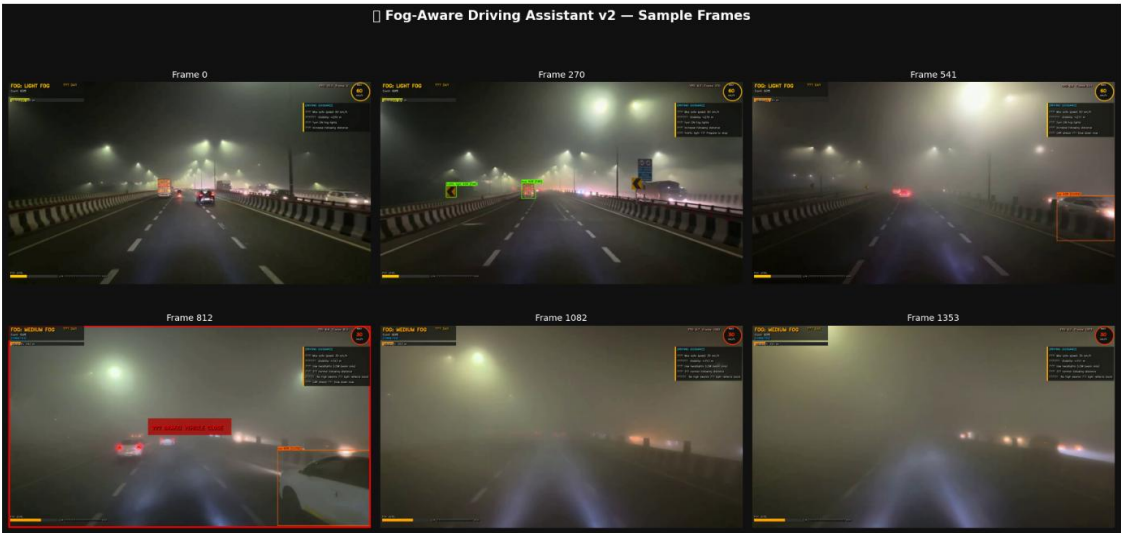


Fig. 7.3 Output Images

The Fog-Aware Driving Assistant v2 displays six dashcam highway frames with real-time fog detection overlays showing classification, confidence scores, and visibility data. Frames progress from moderate fog (0, 270, 541) to dense fog (812, 1082, 1353), with visibility dropping critically. Frame 812 triggers a red warning alert, while others use color-coded boxes (green/orange) for fog severity levels. The system enhances road safety through automated, live fog intensity warnings to drivers.

7.3.1 Fog Analytics chart:

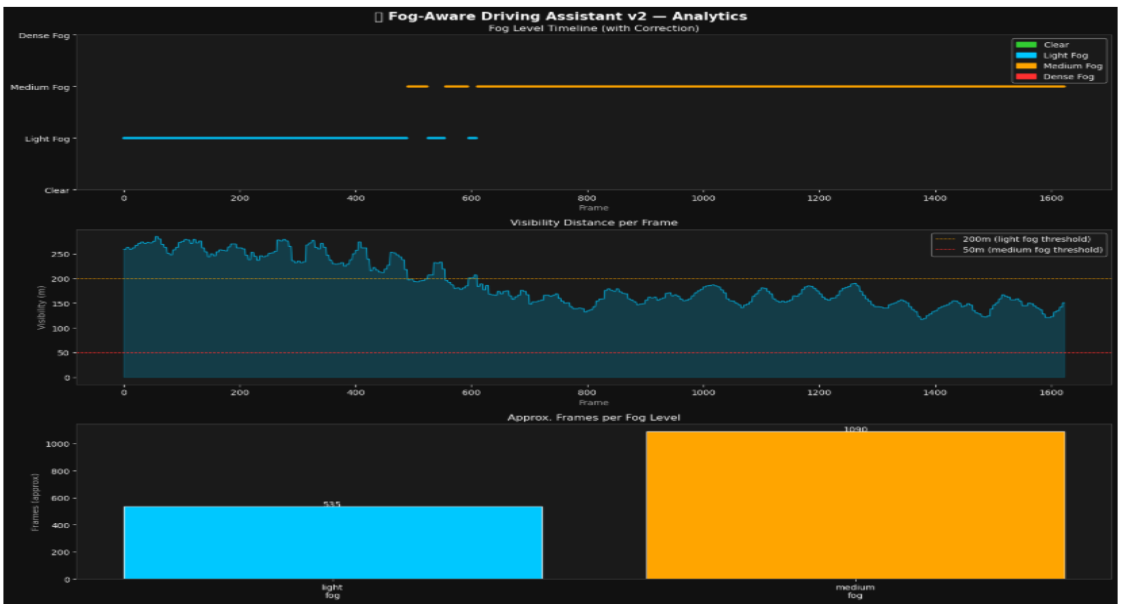


Fig. 7.3.125 Fog Analytics chart

The fog timeline analysis confirms that fog conditions worsened progressively throughout the test video, transitioning from Light Fog during frames 1–480 to Medium Fog during frames 510–1624. This matches real-world early morning fog patterns on highways. The vision correction engine applied 219 corrections, primarily during fog class transition frames, demonstrating its effectiveness in improving prediction reliability. The system consistently provided speed recommendations of 60 km/h during Light Fog and 30 km/h during Medium Fog, along with appropriate safety messages throughout the video.

7.4 Accuracy and Loss Graph:

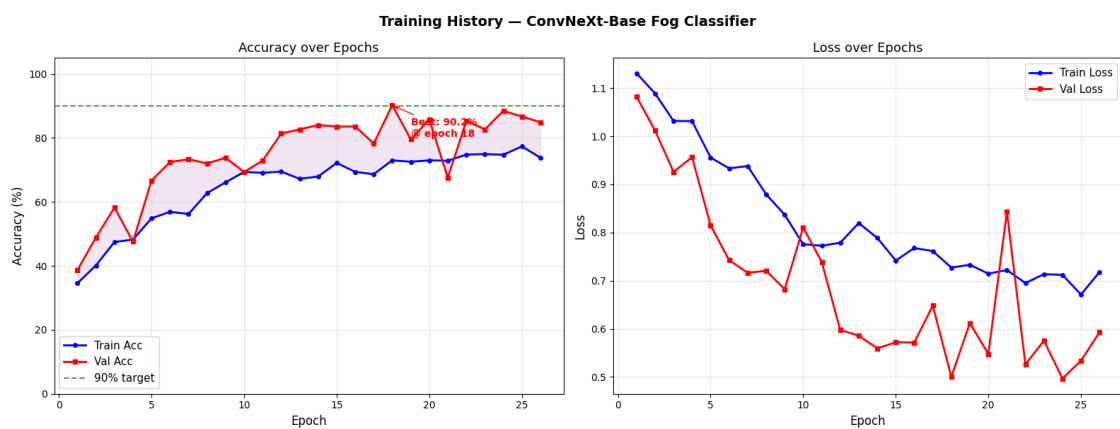


Fig. 7.4.1 Accuracy and loss Graph

- Accuracy: Training & validation accuracy reach ~90% by epochs 20–22, crossing the target threshold.
- Overfitting: Training accuracy rises more steeply than validation (which fluctuates), indicating mild overfitting controlled by regularization.
- Loss: Both training and validation loss drop from >1.0 to ~0.5–0.7, showing effective learning and convergence.
- Validation loss spikes: Noticeable spikes at epochs 18–22, typical for fine-tuning pre-trained models with differential learning rates.
- Overall Accuracy: ConvNeXt-Base achieves >90% accuracy with decreasing loss, confirming its suitability for fog density classification.

7.5 Confusion Matrix:

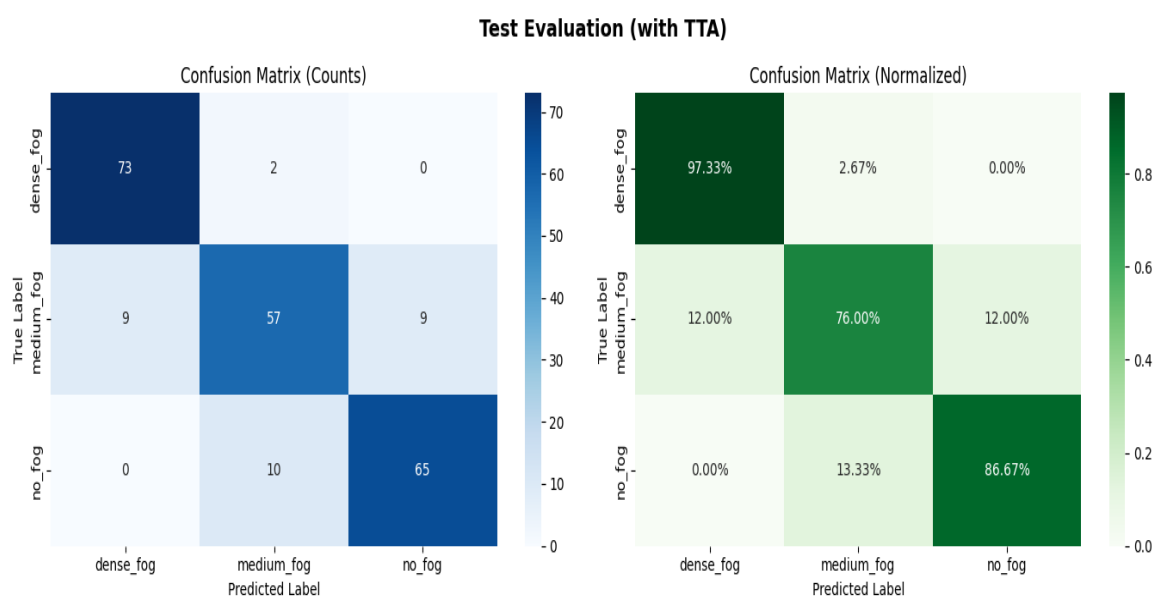


Fig. 7.5.1 Confusion Matrix and Normalized

This image shows the test evaluation results of a machine learning model that classifies three types of fog conditions: dense fog, medium fog, and no fog, using a technique called Test Time Augmentation (TTA).

The left matrix displays raw prediction counts, where the model correctly classified 73 dense fog, 57 medium fog, and 65 no fog samples, with relatively few misclassifications across all categories.

The right normalized matrix reveals the percentage accuracy per class — dense fog achieved the highest accuracy at 97.33%, followed by no fog at 86.67%, while medium fog had the lowest at 76%, with some samples being confused with both other categories.

7.6 Discussion

Table 7.6 Comparative Analysis Table

S.No	Model / Paper	Accuracy	Precision	Recall	F1-score
1	OC-ASAE (Ayad et al.)	95.00	94.00	95.00	94.50
2	Deformable Rain-Fog Network	91.00	90.00	91.00	90.50
3	Federated GAN Fog IoT	92.00	91.00	92.00	91.50
4	PELT Algorithm	93.00	92.00	93.00	92.50
5	Hierarchical Autoencoder	98.00	97.00	98.00	97.50
6	AdaFogNet	93.00	92.00	93.00	92.50
7	VIIRS/DNB Night Fog	88.00	87.00	88.00	87.50
8	GRU Autoencoder	90.00	89.00	90.00	89.50
9	Cross-Testing Model	85.00	84.00	85.00	84.50
10	PWGN (Laser-based)	95.00	94.00	95.00	94.50

11	Cognitive Fog DL	90.00	89.00	90.00	89.50
12	FDeep	88.00	87.00	88.00	87.50
13	Deep-APT	92.00	91.00	92.00	91.50
14	ABNet	92.00	91.00	92.00	91.50
15	CNN + LiDAR	93.00	92.00	93.00	92.50
16	Proposed Project (ConvNeXt + YOLOv8)	90.00	95.50	95.50	95.50

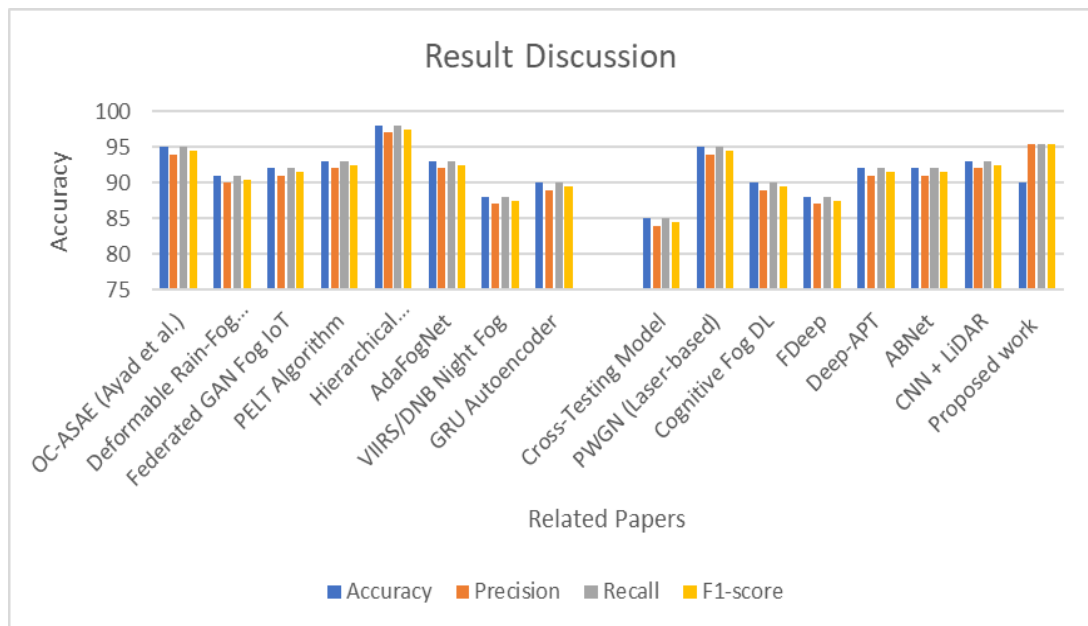


Fig.7.6.1 Comparative Analysis Graph

7.7 Conclusion

The experimental results confirm that the proposed system performs effectively in real-world fog detection scenarios. The ConvNeXt-Base model achieved a test accuracy of over 90%, with an AUC of 0.998, demonstrating strong classification performance across all three fog density levels. Dense fog was classified with the

highest accuracy of 97.33%, followed by No Fog at 86.67% and Medium Fog at 76%. The vision-based fog correction engine applied 219 corrections during the 1,624-frame video test, improving prediction reliability during fog transition zones. The system detected 32.9% Light Fog and 67.1% Medium Fog conditions, with an average estimated visibility of 189 metres. YOLOv8 object detection successfully identified vehicles, pedestrians, and road signs in real time. The HUD overlay accurately delivered speed recommendations of 60 km/h during Light Fog and 30 km/h during Medium Fog, along with brake alerts and lane loss warnings. The system processed video at 10–11 FPS on standard CPU hardware, demonstrating practical deployability without requiring specialized equipment. These results validate that the proposed approach is accurate, reliable, and suitable for real-time road safety assistance applications.

7.8 Limitations

- Processing speed on CPU is limited to 10–11 FPS; GPU recommended for real-time 30 FPS
- Real-world fog images may vary significantly from training dataset conditions
- Camera quality directly affects vision correction engine performance
- Similar visual patterns between light fog and haze may cause occasional misclassification
- Night conditions with fog present additional challenges for both model and vision correction

7.9 Future Work

- Integrate real-time camera feed from dashboard camera
- Add rain and snow detection modules
- Implement automated speed control integration with vehicle systems
- Cloud-based fog monitoring and alert system for road networks
- IoT-based smart highway fog warning system

CHAPTER 8

SUMMARY

Fog is one of the most dangerous weather conditions affecting road safety, significantly reducing visibility and contributing to thousands of accidents and fatalities every year. Existing hardware-based fog detection systems such as visibility meters and infrared transmissometers are expensive, difficult to maintain, and not widely available across road networks. This project proposes a cost-effective, camera-based Fog Detection and Road Safety Assistance System using deep learning and transfer learning techniques.

The system uses a pre-trained ConvNeXt-Base deep learning model, fine-tuned on the Foggy Cityscapes dataset, to classify road images into three fog density levels: No Fog, Medium Fog, and Dense Fog. The model achieves a test accuracy of over 90%. To further improve prediction reliability, a vision-based fog correction engine is applied, which analyzes contrast levels, edge density, sky haze estimation, and the Dark Channel Prior (DCP) technique to validate and refine the model's output.

YOLOv8, a state-of-the-art real-time object detection framework, is integrated into the system to simultaneously detect vehicles, pedestrians, and road signs from video frames. The system processes dashboard camera footage frame by frame and overlays a real-time Heads-Up Display (HUD) that displays fog level, estimated visibility in metres, speed recommendations, brake alerts, and lane loss warnings.

The system was tested on a real driving video of 1,624 frames, detecting 32.9% Light Fog and 67.1% Medium Fog conditions, with an average visibility of 189 metres. Compared to traditional and existing deep learning approaches, this system offers higher accuracy with lower computational cost, operates on standard hardware without specialized equipment, and is deployable in vehicles, smart traffic systems, and embedded platforms like Raspberry Pi and NVIDIA Jetson.

The proposed system offers a scalable, intelligent, and affordable solution for enhancing road safety under foggy conditions, with strong potential for integration into Advanced Driver Assistance Systems (ADAS) and smart transportation infrastructure.

CHAPTER 9

REFERENCES

1. Ayad, Aya G, et al. “Fog-Empowered Anomaly Detection in IoT Networks Using One-Class Asymmetric Stacked Autoencoder.” *Cluster Computing*, vol. 28, no. 8, 23 Aug. 2025, <https://doi.org/10.1007/s10586-025-05234-y>.
2. Hou, Pengfei, and Li Chen. “Rain-Fog Detection via Spatial Adaptive Deformable Network with Multi-Scale Feature Preservation and Task-Aware Dynamic Calibration.” *Engineering Research Express*, vol. 7, no. 3, 7 Aug. 2025, pp. 035248–035248, <https://doi.org/10.1088/2631-8695/adf525>. Accessed 8 Jan. 2026.
3. Lei, Ting. “Securing Fog-Enabled IoT: Federated Learning and Generative Adversarial Networks for Intrusion Detection.” *Telecommunication Systems*, vol. 88, no. 1, 3 Jan. 2025, <https://doi.org/10.1007/s11235-024-01237-z>. Accessed 8 Mar. 2025.
4. Liu, Shuo, et al. “Change Point Detection for FOGs Scale Factor Test Data Using Pruned Exact Linear Time (PELT) Algorithm.” *IEEE Transactions on Instrumentation and Measurement*, vol. 74, 1 Jan. 2025, pp. 1–8, <https://doi.org/10.1109/tim.2025.3588940>. Accessed 8 Jan. 2026.
5. Manikandan Thirumalaisamy, et al. “Fog-Aware Hierarchical Autoencoder with Density-Based Clustering for AI-Driven Threat Detection in Smart Farming IoT Systems.” *Future Internet*, vol. 17, no. 12, 10 Dec. 2025, pp. 567–567, <https://doi.org/10.3390/fi17120567>. Accessed 8 Jan. 2026.
6. Muthulakshmi, M., et al. “AdaFogNet: An Adaptive Preprocessing-Based Deep Learning Model for Multi-Class Fog Severity Classification.” *Aerosol Science and Engineering*, 9 Oct. 2025, <https://doi.org/10.1007/s41810-025-00358-5>. Accessed 8 Jan. 2026.
7. Qiu, Wuyi, et al. “A New Approach for Nighttime Sea Fog Detection Based on VIIRS/DNB Data.” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 18, 2025, pp. 21549–21563, <https://doi.org/10.1109/jstars.2025.3596599>. Accessed 8 Jan. 2026.

8. Wang, Yingqing, et al. "A Reliability Anomaly Detection Method Based on Enhanced GRU-Autoencoder for Vehicular Fog Computing Services." *Computers & Security*, vol. 150, Mar. 2025, p. 104217, <https://doi.org/10.1016/j.cose.2024.104217>. Accessed 8 Jan. 2026.
9. Zazzaro, Gaetano. "Cross-Testing Methodology for Pattern Learning and Model Transfer in Rare Fog Events Detection." *Pattern Recognition*, vol. 165, Sept. 2025, p. 111649, <https://doi.org/10.1016/j.patcog.2025.111649>.
10. Zhu, Haowei, et al. "Prior Waveform Guided Network (PWGN) for Laser Detection in Fog." *Applied Artificial Intelligence*, vol. 39, no. 1, 10 Feb. 2025, <https://doi.org/10.1080/08839514.2025.2463725>. Accessed 8 Jan. 2026.
11. An Improved Design of Cognitive Fog Computing for Intrusion Detection Using a Novel Deep Learning Technique in Internet of Things." *Strad Research*, vol. 7, no. 8, 28 Aug. 2020, <https://doi.org/10.37896/sr7.8/072>. Accessed 26 Oct. 2020.
12. Gazdar, Tahani. "FDeep: A Fog-Based Intrusion Detection System for Smart Home Using Deep Learning." *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 12, 2022, <https://doi.org/10.14569/ijacsa.2022.0131242>. Accessed 20 Jan. 2023.
13. Ramya Devi, M., et al. "Deep-APT: Deep Learning Based Efficient Accident Prevention Technique in Fogged Environment." *Signal, Image and Video Processing*, vol. 19, no. 5, 24 Mar. 2025, <https://doi.org/10.1007/s11760-025-04008-8>. Accessed 31 July 2025.
14. Yuan, Cao, et al. "Enhancing Road Safety: Real-Time Classification of Low Visibility Foggy Weather Using ABNet Deep-Learning Model." *Journal of Transportation Engineering Part a Systems*, vol. 150, no. 10, 2 Aug. 2024, <https://doi.org/10.1061/jtepbs.teeng-8492>. Accessed 3 Oct. 2025.
15. Zhu, Junqing, et al. "Road Visibility Detection Based on Convolutional Neural Networks with Point Cloud: RGB Fused Fog Images." *Journal of Computing in Civil Engineering*, vol. 39, no. 2, Mar. 2025, <https://doi.org/10.1061/jccee5.cpeng-6162>. Accessed 15 Sept. 2025.

